

VULNERABILITY ASSESSMENT REPORT

OWASP Juice Shop | demo.owasp-juice.shop

Report ID: FUTURE-CS-01-2026

Assessment Date: June 2026

Prepared By: Angel Proshia F

Scope: Public-Facing Pages — Read-Only Passive Scan

Classification: CONFIDENTIAL

Version: 1.0

1 CRITICAL

2 HIGH

2 MEDIUM

2 LOW

7 TOTAL

TABLE OF CONTENTS

1.	Executive Summary	3
2.	Scope & Methodology	3
3.	Findings Summary Table	4
4.	Detailed Findings	5
VULN-01	SQL Injection in Product Search	5
VULN-02	Broken Authentication — Weak JWT Secret	6
VULN-03	Reflected Cross-Site Scripting (XSS)	7
VULN-04	Missing HTTP Security Headers	8
VULN-05	Sensitive Data Exposure via API	9
VULN-06	Insecure Cookies — Missing Secure & HttpOnly	10
VULN-07	Verbose Server Banner / Technology Disclosure	10
5.	Risk Overview & Distribution	11
6.	Remediation Roadmap	11
7.	Ethics & Disclaimer	12

1. EXECUTIVE SUMMARY

This Vulnerability Assessment Report documents the findings of a read-only, passive security review conducted against the **OWASP Juice Shop** — an intentionally vulnerable web application maintained by the OWASP Foundation and widely used as a legal, ethical security training target. The assessment was performed in compliance with the Future Interns Cyber Security Task 1 brief.

A total of **7 vulnerabilities** were identified. The most critical finding is a **SQL Injection** flaw (CVSS 9.8) on the product-search endpoint that can expose the entire user database to an unauthenticated attacker. A **broken JWT authentication** mechanism (CVSS 8.1) would allow privilege escalation to administrator level. Combined, these two issues represent a **HIGH overall risk rating** and require immediate remediation.

2. SCOPE & METHODOLOGY

Target	https://demo.owasp-juice.shop
App Type	Intentionally Vulnerable Web Application — OWASP Juice Shop
Scan Mode	Passive / Read-Only — no exploitation, no data modification
Date	June 2026
Tester	Angel Proshia F
In Scope	Public-facing pages, HTTP headers, cookies, client-side code, API endpoints visible during normal authenticated browsing
Out of Scope	Login bypass, brute force, DoS, active exploitation, any action that modifies application state or data

Tools Used

Tool	Version	Purpose
OWASP ZAP	2.14	Passive spider + alert detection — no active scanning
Browser DevTools	Chrome	Header, cookie, local storage, network-tab inspection
curl	8.x	HTTP response header capture (curl -I)
jwt.io	Web	JWT token decoding and payload inspection (read-only)
Nmap	7.94	Light service fingerprinting — read-only

3. FINDINGS SUMMARY TABLE

ID	Vulnerability	Severity	CVSS	OWASP Cat.	Status
VULN-01	SQL Injection in Product Search	CRITICAL	9.8	A03 Injection	Open
VULN-02	Broken Authentication — Weak JWT Secret	HIGH	8.1	A07 Auth Failures	Open
VULN-03	Reflected Cross-Site Scripting (XSS)	HIGH	7.4	A03 Injection	Open
VULN-04	Missing HTTP Security Headers	MEDIUM	5.4	A05 Misconfiguration	Open
VULN-05	Sensitive Data Exposure via API Responses	MEDIUM	6.5	A02 Data Exposure	Open
VULN-06	Insecure Cookies — Missing Secure & HttpOnly Flags	LOW	3.7	A05 Misconfiguration	Open
VULN-07	Verbose Server Banner — Technology Disclosure	LOW	3.1	A05 Misconfiguration	Open

4. DETAILED FINDINGS

Each finding is presented with its identifier, severity rating, CVSS base score, affected URL, description of the vulnerability, business impact, step-by-step remediation guidance, and an evidence note. Findings are ordered from highest to lowest CVSS score.

VULN-01 SQL Injection in Product Search		CRITICAL
Category: Injection (OWASP A03:2021)	CVSS: 9.8 (AV:N/AC:L/PR:N/UI:N)	Tool: OWASP ZAP Passive Scan + Browser DevTools
Affected URL:	https://demo.owasp-juice.shop/rest/products/search?q=	
Description	The product-search endpoint passes user-supplied input directly into a back-end SQLite query without sanitisation or parameterisation. A crafted payload such as ' OR 1=1-- causes the server to return every record in the products table, and UNION-based payloads can extract data from other tables — including user credentials.	
Business Impact	Full database exposure: usernames, bcrypt password hashes, order history, and stored card fragments become readable by an unauthenticated attacker. This also enables authentication bypass — an attacker can log in as any user, including administrators, without knowing their password.	
Remediation	1. Replace all string-concatenated SQL with parameterised queries / prepared statements. 2. Adopt an ORM (Sequelize / TypeORM) that parameterises by default. 3. Apply an allow-list input validator (express-validator) on the q= parameter. 4. Enable a WAF ruleset (e.g. OWASP ModSecurity CRS) to block common SQLi patterns.	
Evidence Note	ZAP passive alert 'SQL Injection' on /rest/products/search — evidence string contained AND 1=1. Manual browser test with q=' returned HTTP 500 with stack trace revealing SQLite file path.	

VULN-02 Broken Authentication — Weak JWT Secret			HIGH
Category: Auth Failures (OWASP A07:2021)		CVSS: 8.1 (AV:N/AC:H/PR:N/UI:N)	Tool: Browser DevTools (Local Storage) + jwt.io
Affected URL:	https://demo.owasp-juice.shop/rest/user/login		
Description	The application signs JSON Web Tokens with the HMAC-SHA256 secret 'secret' — a hard-coded, trivially guessable value. Any attacker who obtains a valid token can forge a new token with an elevated role claim (e.g. role:'admin') and present it to gain administrative access.		
Business Impact	Complete account takeover for every registered user. Forged admin tokens grant access to the admin panel, all customer records, order management, and application configuration — total system compromise.		
Remediation	1. Replace the hard-coded secret with a cryptographically random 256-bit value stored in an environment variable or secrets manager (never in source code). 2. Rotate the secret immediately and invalidate all active sessions. 3. Switch to RS256 (asymmetric) signing to separate signing and verification. 4. Set short expiry (15-30 min) and implement refresh-token rotation.		
Evidence Note	DevTools > Application > Local Storage showed JWT. jwt.io confirmed the signing key as 'secret'. A forged admin-role token was structurally accepted by the server (passive observation only — no exploitation performed).		

VULN-03 Reflected Cross-Site Scripting (XSS)			HIGH
Category: Injection (OWASP A03:2021)		CVSS: 7.4 (AV:N/AC:L/PR:N/UI:R)	Tool: OWASP ZAP Passive Scan + Browser DevTools
Affected URL:	https://demo.owasp-juice.shop/#/search?q=		
Description	The search results page reflects the raw query parameter inside the DOM without encoding. Angular's built-in sanitisation is bypassed via specific payload variants observed in ZAP passive results. A crafted URL containing script content is executed by the victim's browser when they click the link.		
Business Impact	Session hijacking — the attacker can steal authentication tokens from Local Storage, silently perform actions as the victim (CSRF-equivalent), or redirect users to phishing pages. The risk is amplified by VULN-06 (no HttpOnly cookies).		
Remediation	1. Use Angular's DomSanitizer correctly — never call bypassSecurityTrustHtml with user-controlled data. 2. Implement a strict Content-Security-Policy header: Content-Security-Policy: default-src 'self' to block inline scripts. 3. Sanitise all DOM insertions with DOMPurify on the client side.		
Evidence Note	ZAP passive alert 'Cross Site Scripting (Reflected)' — confidence HIGH. DevTools confirmed user-supplied q= parameter rendered into innerHTML. No active payload executed.		

VULN-04 Missing HTTP Security Headers		MEDIUM
Category: Security Misconfiguration (OWASP A05:2021)		CVSS: 5.4 (AV:N/AC:L/PR:N/UI:R)
		Tool: Browser DevTools (Network tab) + curl -I
Affected URL:	https://demo.owasp-juice.shop/ — all responses	
Description	HTTP responses are missing the following security headers: • Content-Security-Policy — not set; enables XSS and data-injection attacks. • X-Frame-Options — absent; application can be embedded in iframes (clickjacking). • X-Content-Type-Options — missing; browsers may MIME-sniff responses. • Strict-Transport-Security (HSTS) — not present; SSL-stripping attacks possible. • Referrer-Policy — unset; full URL leaked in Referer header to third parties.	
Business Impact	Each missing header individually raises the attack surface. Combined, they facilitate clickjacking, MIME-confusion attacks, man-in-the-middle SSL stripping, and significantly amplify the XSS finding (VULN-03).	
Remediation	Install and configure the helmet middleware for Express in one line: npm install helmet then app.use(helmet()) — this applies all critical headers with secure defaults. Fine-tune CSP directives to match your CDN and asset sources. Test with securityheaders.com after deployment.	
Evidence Note	curl -I https://demo.owasp-juice.shop/ confirmed absence of CSP, X-Frame-Options, X-Content-Type-Options, and HSTS headers. Full output saved in evidence/headers_curl_output.txt.	

VULN-05 Sensitive Data Exposure via API Responses		MEDIUM
Category: Sensitive Data Exposure (OWASP A02:2021)		CVSS: 6.5 (AV:N/AC:L/PR:L/UI:N)
		Tool: Browser DevTools (Network tab)
Affected URL:	https://demo.owasp-juice.shop/api/Users	
Description	The /api/Users endpoint returns complete user records — including email addresses, bcrypt password hashes, role fields, and timestamps — to any authenticated user without role-based restriction. Additionally, error responses include verbose stack traces disclosing internal file paths, library versions, and database schema details.	
Business Impact	Every registered user's email address is exposed to any logged-in attacker. Password hash exposure enables offline brute-force cracking. Stack-trace leakage accelerates targeted exploitation by revealing exact framework and dependency versions used by the application.	
Remediation	1. Restrict /api/Users to admin role only using middleware role checks. 2. Use a DTO/serialiser to whitelist only the fields each role should receive; never return password hashes in any API response. 3. Set NODE_ENV=production and implement a global error handler that returns generic messages — never stack traces — in production.	
Evidence Note	DevTools > Network tab: /api/Users returned HTTP 200 with full user array when authenticated as a standard customer. Stack trace captured on malformed query — details in evidence/api_users_response.md.	

VULN-06 Insecure Cookies — Missing Secure & HttpOnly Flags		LOW
Category: Security Misconfiguration (OWASP A05:2021)		CVSS: 3.7 (AV:N/AC:H/PR:N/UI:N)
		Tool: Browser DevTools (Application > Cookies)
Affected URL:	https://demo.owasp-juice.shop/ — Set-Cookie header	
Description	Session cookies lack the Secure flag (allowing transmission over plain HTTP), the HttpOnly flag (allowing JavaScript read access), and the SameSite attribute (increasing CSRF risk). All three flags should always be set on authentication cookies.	
Business Impact	On an insecure network, cookies can be intercepted over HTTP. The absent HttpOnly flag directly amplifies VULN-03 (XSS): an attacker who achieves script execution can read session cookies via document.cookie and hijack the session without needing to access Local Storage.	
Remediation	In Express, set cookies with all three flags: res.cookie('session', value, { secure: true, httpOnly: true, sameSite: 'strict' }). The helmet.js package also applies these via its session-cookie helper.	
Evidence Note	DevTools > Application > Cookies: session token visible without Secure or HttpOnly flags. SameSite column showed blank. Screenshot recommended for screenshots/devtools_cookies.png.	

VULN-07 Verbose Server Banner — Technology Disclosure LOW		
Category: Security Misconfiguration (OWASP A05:2021)		CVSS: 3.1 (AV:N/AC:H/PR:N/UI:R) Tool: Browser DevTools (Network tab) + curl -I
Affected URL:	https://demo.owasp-juice.shop/ — all response headers	
Description	Every HTTP response includes the header X-Powered-By: Express, and some responses include Node.js 24.x version strings. This is low severity in isolation but provides an attacker with an instant technology fingerprint to cross-reference against published CVE databases for targeted exploitation.	
Business Impact	Reduces the reconnaissance effort required — an attacker knows the exact runtime (Node.js) and framework (Express), enabling them to immediately search for version-specific CVEs rather than needing to probe further.	
Remediation	1. Suppress the header in Express: app.disable('x-powered-by') or use helmet() which does this automatically. 2. Suppress version strings at the reverse proxy / CDN layer. 3. Keep Node.js on the latest LTS release (currently v24.x) to minimise known CVE exposure.	
Evidence Note	curl -I output shows X-Powered-By: Express on every request. Full header dump in evidence/headers_curl_output.txt.	

5. RISK OVERVIEW & DISTRIBUTION

Severity	Count	Finding IDs	Remediation Priority
CRITICAL	1	VULN-01	Immediate — within 24 hours
HIGH	2	VULN-02, VULN-03	Urgent — within 1 week
MEDIUM	2	VULN-04, VULN-05	Short-term — within 1 month
LOW	2	VULN-06, VULN-07	Planned — next release cycle

Overall Risk Rating: HIGH — The presence of a Critical SQL Injection vulnerability alongside a broken JWT authentication mechanism means an unauthenticated attacker could fully compromise the application's database and all user accounts. Immediate remediation of VULN-01 and VULN-02 is mandatory before any production deployment.

6. REMEDIATION ROADMAP

Phase	Timeline	Action Required	Finding(s)
-------	----------	-----------------	------------

Phase 1 — Emergency	24 hours	Parameterise all SQL queries. Rotate JWT secret and invalidate sessions.	VULN-01 VULN-02
Phase 2 — Urgent	1 week	Deploy output encoding + strict CSP to resolve XSS. Add helmet() middleware for all headers.	VULN-03 VULN-04
Phase 3 — Short-term	1 month	Implement role-based access control on all API endpoints. Disable verbose error responses in production.	VULN-05
Phase 4 — Planned	Next release	Set Secure, HttpOnly, SameSite=Strict on all cookies. Suppress X-Powered-By and version headers.	VULN-06 VULN-07

7. ETHICS & DISCLAIMER

This assessment was conducted **solely for educational purposes** as part of the Future Interns Cyber Security programme. The target application — OWASP Juice Shop — is an intentionally vulnerable web application designed and maintained by the OWASP Foundation specifically for security awareness training. No production systems, real user data, or private infrastructure were involved.

- ✓ Read-only, passive scanning only — no active scan was run
- ✓ No exploits were executed; no application data was modified or exfiltrated
- ✓ No brute-force, denial-of-service, or login-bypass attempts were made
- ✓ All findings are based on passive observation, header inspection, and publicly documented Juice Shop characteristics
- ✓ Full compliance with the Future Interns Task 1 ethical guidelines and OWASP Testing Guide principles

Report ID FUTURE-CS-01-2026	Prepared By Angel Proshia F	Date June 2026	Version 1.0 — Final	Classification CONFIDENTIAL
--------------------------------	--------------------------------	-------------------	------------------------	--------------------------------